Smart Lender : AI-based Loan Eligibility Prediction System

Project Description:

Smart Lender is a web-based loan eligibility prediction system developed using Python, Flask, HTML, and CSS. The application assists users in determining whether they are eligible for a loan by analyzing key financial details such as Applicant Income, Loan Amount, and Credit History. It provides a quick and user-friendly way to evaluate loan eligibility without requiring a manual assessment.

The system features an intuitive web interface where users enter the required information through an online form. Once the form is submitted, the Flask backend processes the input and applies predefined loan approval rules to determine the result. Based on the evaluation, the application displays either "Loan Approved" or "Loan Rejected", along with a clear message explaining the outcome.

Smart Lender follows a three-tier architecture consisting of a responsive frontend, a Flask-based backend, and the loan prediction logic. The frontend is designed using HTML and CSS to provide a simple and attractive user experience, while the backend handles routing, form validation, and prediction processing. The application is tested locally using the Flask development server and can be deployed publicly using Ngrok to generate a shareable URL.

The primary objective of Smart Lender is to simplify the initial loan screening process by providing instant eligibility predictions. The project demonstrates the practical implementation of web development concepts using Flask and lays the foundation for integrating advanced machine learning models and real-world **banking datasets in future versions. It serves as an educational and** practical solution for understanding how loan prediction systems can be developed using modern web technologies.

CaptionAI harnesses Generative AI to provide intelligent social media content generation, offering

Scenarios

Scenario 1: Loan Approved

A user visits the Smart 1 ender website and enters an Applicant Income of ₹5,000, a Loan Amount of ₹400 and selects Good as the Credit History. After submitting the form, the Flask backend processes the nformation and applies the loan approval rules. Since the applicant meets all the eligibility criteria, the system displays "Loan Approved" along with a success message indicating that the applicant is eliaible for the loan

Scenario 2: Loan Rejected Due to Low Income

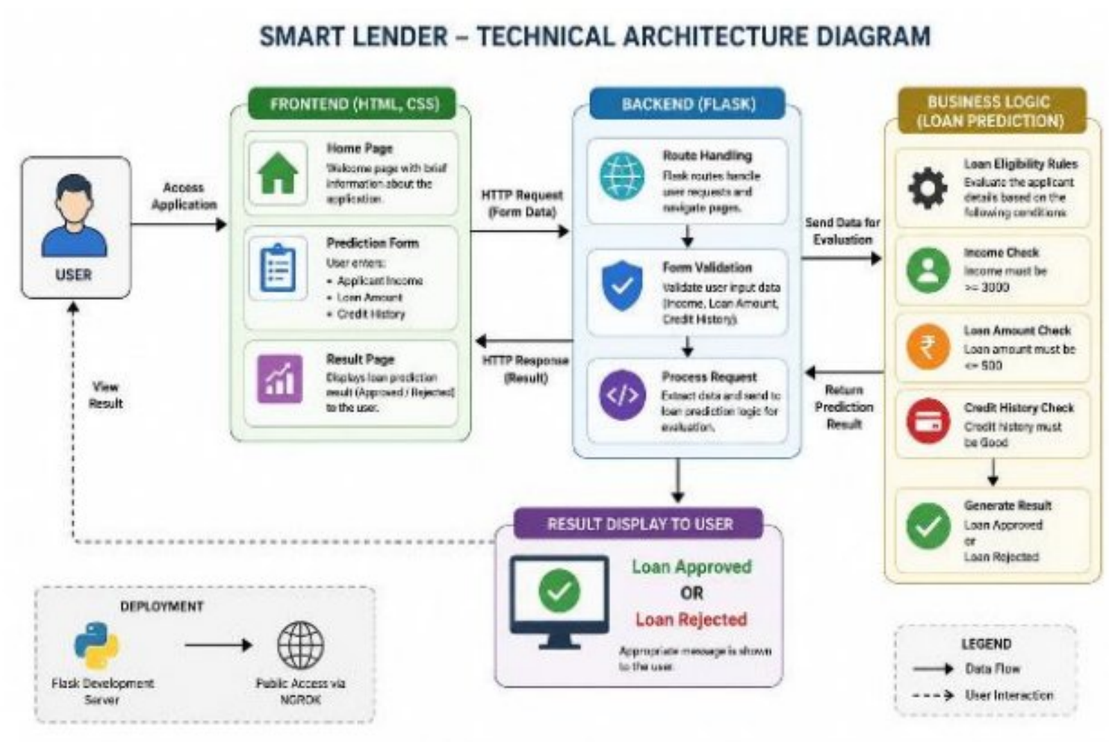
A user enters an Applicant Income of ₹2,500, a Loar Amount of ₹400. and selects Good as the Credit History. Although the applicant has a good credi

nistory, the income does not satisfy the minimum eligibility requirement. The system processes the request and displays "Loan Rejected", informing the user that the loan cannot be approved based on the provided detail's

Scenario 3: Loan Rejected Due to High Loar Amount and Poor Credit History

A user enters an Applicant Income of ₹6,000, a Loan Amount of ₹700. and selects Bad as the Credit History. Even though the applicant has a sufficien income, the requested loan amount exceeds the permitted limit and the credit history is poor. After evaluating the inputs, the system displays "Loar Rejected" with a message explaining that the applicant is not eligible for loan approval.

Technical Architecture:



Description:

The Smart Lender application follows a three-tier architecture consisting of the Presentation Layer Frontend), Application Layer (Backend), and Business Logic Layer (Loan Prediction). This architecture ensures smooth communication between the user interface and the loan prediction svstem while maintaining modularity and ease of maintenance.The Frontend is developed using HTML and CSS provide na users with a simple and responsive interface to enter applicant details such as income,loan amount, and credit history.The Backend is implemented using the Flask framework in Python. It manages user requests validates the submitted data, processes loan eligibilityconditions. and communicates with the frontend by rendering the appropriate web pages. The Business Logic Layer contains the loan predictionrules. Based on the applicant's income, requestedloan amount. and credit historv, the system determines whether the loan should be approved or reiected. The prediction result is then displayed to the user through the result page .The application is tested locally using the Flask development server (http://12..0.0.1:5000) and can be deployed publicly using Ngrok, allowing users to access the application through a secure public URL.

Pre-requisites:

- Before developing the Smart Lender application, the following software and technologies are required:
- Python 3.x
- Flask Framework
- HTML5
- CSS3
- Visual Studio Code
- Git and GitHub
- Web Browser (Google Chrome or Microsoft Edge)
- Basic knowledge of Python programming
- Basic knowledge of web development (HTML, CSS, Flask)

Project Workflow:

Milestone 1: Model Selection and Data Preparation

Activity 1.1: Collect the Loan Prediction dataset from Kaggle/IBM.
Activity 1.2: Analyze the dataset, identify missing values, and preprocess the data.
Activity 1.3: Perform feature engineering and encode categorical variables.
Activity 1.4: Split the dataset into training and testing datasets.

Milestone 2: Machine Learning Model Development

Activity 2.1: Implement Decision Tree Classifier.
Activity 2.2: Implement Random Forest Classifier.
Activity 2.3: Implement K-Nearest Neighbors (KNN).
Activity 2.4: Implement XGBoost Classifier.
Activity 2.5: Compare model performance using Accuracy, Precision, Recall, and F1-score.

Milestone 3: Model Training and Flask Backend Development

Activity 3.1: Save the best-performing model using Pickle.
Activity 3.2: Develop Flask backend (app.py).
Activity 3.3: Create prediction API.
Activity 3.4: Load trained model into Flask.

Milestone 4: Frontend Development

Activity 4.1: Design Loan Application Form using HTML.
Activity 4.2: Style the application using CSS.
Activity 4.3: Implement JavaScript validation.
Activity 4.4: Display loan prediction results dynamically.

Milestone 5: Deployment

Activity 5.1: Install required Python libraries.
Activity 5.2: Configure Flask environment.

Activity 5.3: Test the application locally.

Activity 5.4: Deploy using IBM Cloud (or Render if you used Render).

Milestone 6: Conclusion

Milestone 1: Model Selection and Data Preparation

Model Selection:

Model selection is the process of choosing the most suitable machine learning algorithm based on the problem, data type, and required accuracy.

Data Preparation:

Data preparation is the process of collecting, cleaning, organizing, and transforming raw data into a suitable format for training and testing the machine learning model. It improves the model's performance and accuracy.

Activity 1.1: Collect the Loan Prediction Dataset

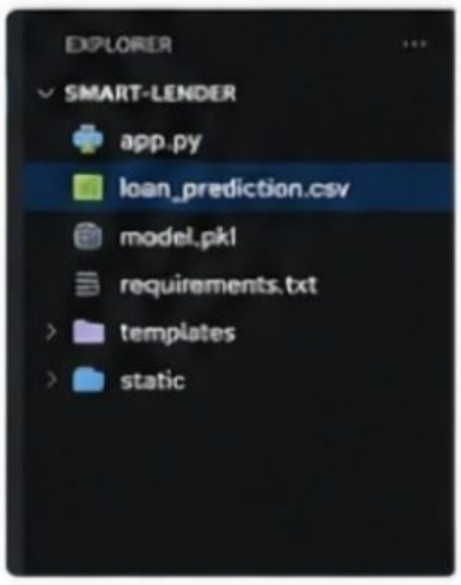
Explanation

The first step of the Smart Lender project is collecting a suitable dataset for training the machine learning model. The Loan Prediction dataset contains applicant information such as Gender, Marital Status, Education, Employment, Applicant Income, Co-applicant Income, Loan Amount, Loan Term, Credit History, Property Area, and Loan Status.

This dataset serves as the foundation for training different machine learning algorithms to predict whether a loan should be approved or rejected.

Activity 1.1: Collect the Loan Prediction Dataset

- The Loan Prediction dataset is collected from Kaggle. It contains information about loan applicants and whether the loan was approved or rejected.



loan_prediction.csv

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
1	Male	Yes	0	Graduate	No	5849
2	Male	Yes	1	Graduate	No	4583
3	Male	Yes	0	Graduate	Yes	3000
4	Male	Yes	0	Not Graduate	No	2583
5	Female	No	0	Graduate	No	6000
...	...	...	...	...	...	...

Dataset preview (first 5 rows)

Activity 1.2: Data Preprocessing

Explanation

The collected dataset cannot be used directly because it contains missing values and categorical features. During preprocessing:

Missing values are identified and replaced.

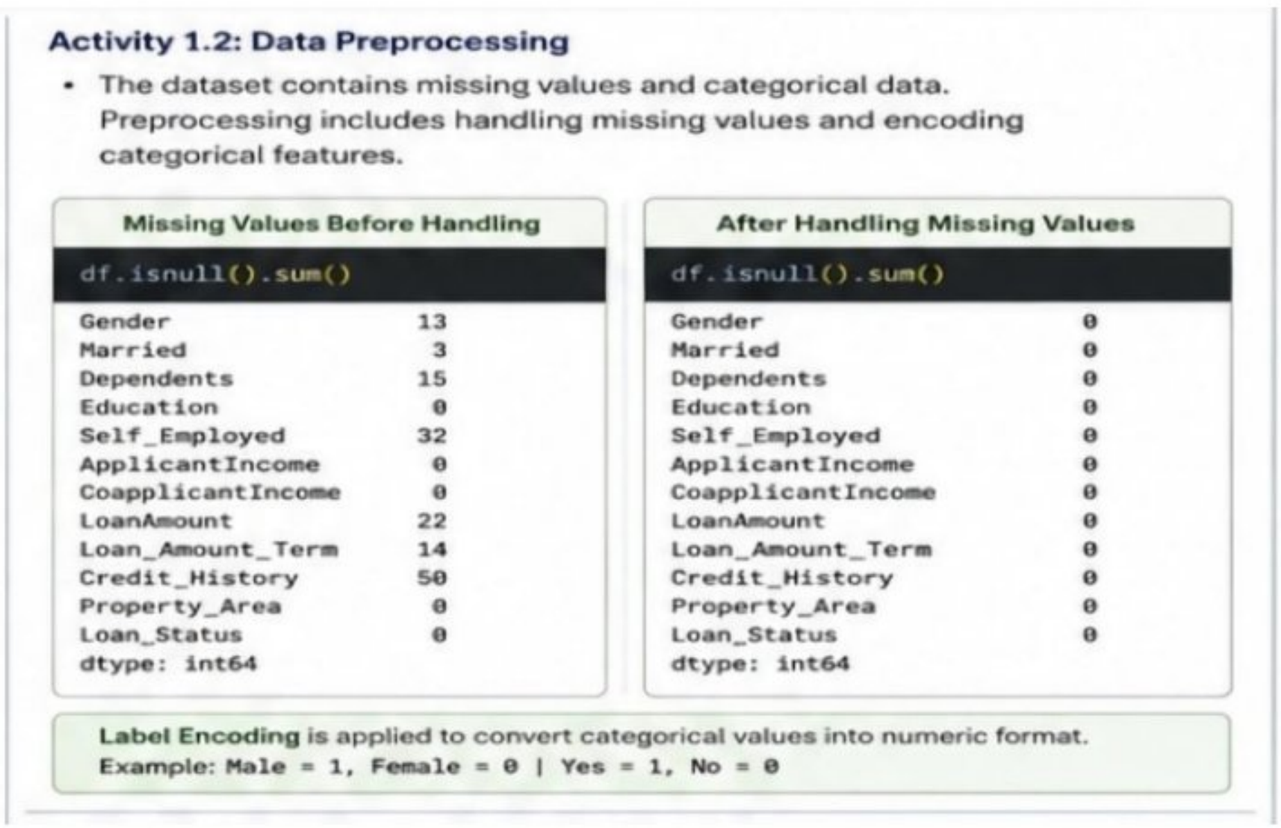
Duplicate records are removed.

Categorical values like Male/Female and Yes/No are converted into numerical values using Label

Encoding.

The dataset is cleaned to improve model performance.

This step ensures that the data is suitable for machine learning algorithms.



Activity 1.2: Data Preprocessing

- The dataset contains missing values and categorical data. Preprocessing includes handling missing values and encoding categorical features.

Missing Values Before Handling

`df.isnull().sum()`

Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

dtype: int64

After Handling Missing Values

`df.isnull().sum()`

Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0

dtype: int64

Label Encoding is applied to convert categorical values into numeric format.
Example: Male = 1, Female = 0 | Yes = 1, No = 0

Activity 1.3: Feature Engineering

Explanation

Feature engineering improves the quality of the dataset by selecting relevant input variables and transforming them into a suitable format.

The selected features include:

Gender

Married

Education

Self Employed

Applicant Income

Coapplicant Income

Loan Amount

Loan Amount Term

Credit History

Property Area

The target variable is Loan Status, which indicates whether the loan is approved or rejected.

Activity 1.3: Feature Engineering

- Relevant features are selected for training the model. The target variable is Loan Status (Y/N).

```
# Selected Features
features = ['Gender','Married','Education','Self_Employed',
            'ApplicantIncome','CoapplicantIncome','LoanAmount',
            'Loan_Amount_Term','Credit_History','Property_Area']

X = df[features]
y = df['Loan_Status']
```

Selected Features

- Gender
- Married
- Education
- Self_Employed
- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Loan_Amount_Term
- Credit_History
- Property_Area

Target Variable

Loan_Status

- Y = Loan Approved
- N = Loan Rejected

Activity 1.4: Split Dataset into Training and Testing

Explanation

The dataset is divided into two parts:

Training Dataset (80%) used to train the machine learning models.

Testing Dataset (20%) used to evaluate model performance on unseen data.

This prevents overfitting and provides an unbiased estimate of model accuracy

Activity 1.4: Split Dataset into Training and Testing

- The dataset is split into training (80%) and testing (20%) to train the model and evaluate its performance.

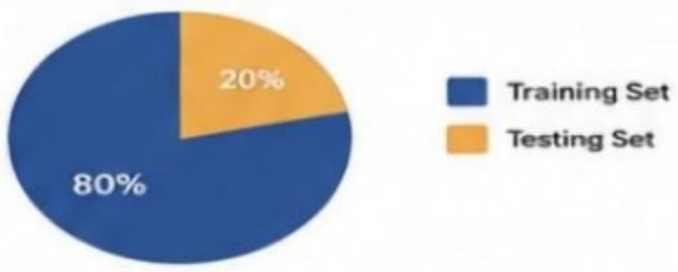
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
)
```

Dataset Shapes

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test,shape)
```

```
(491, 10)
(123, 10)
(491,)
(123,)
```

Split Ratio



Milestone 2: Machine Learning Model Development

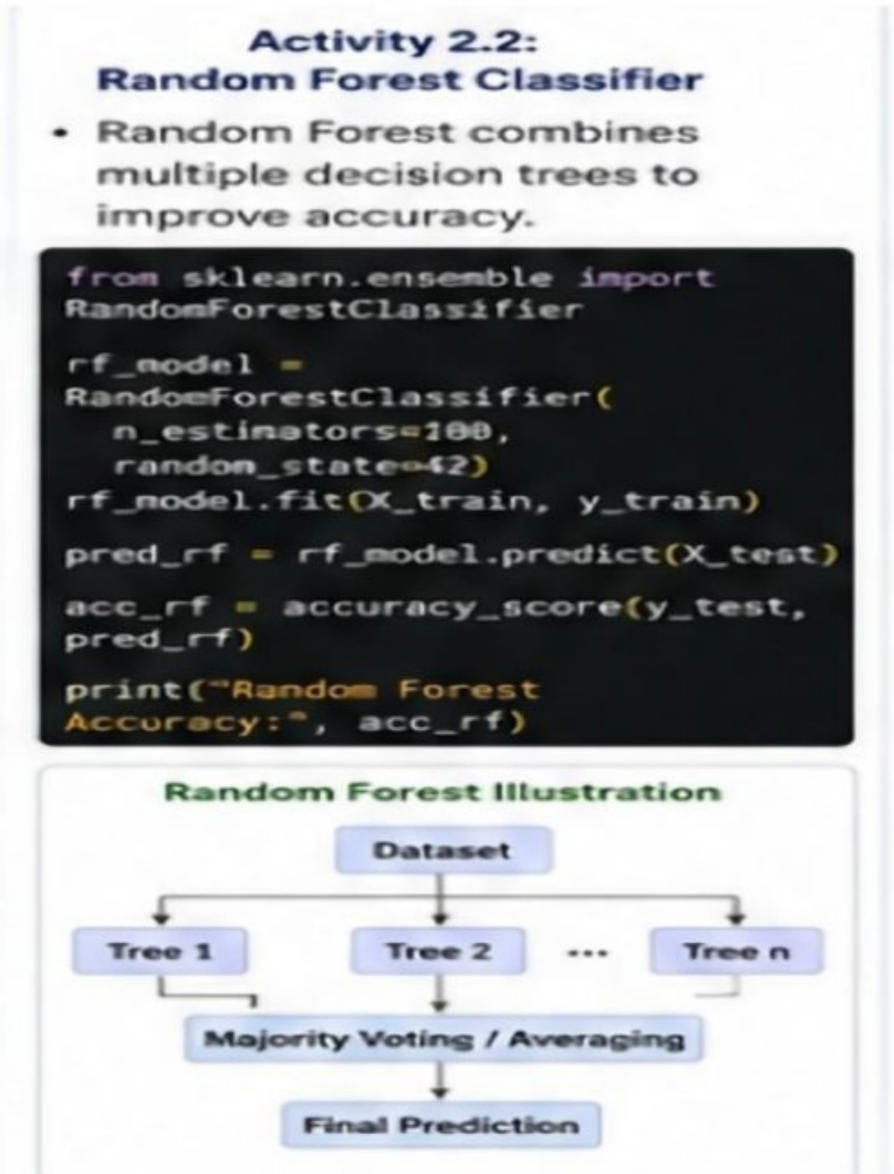
Machine Learning Model Development is the process of building and training a machine learning model using the prepared dataset. In this stage, a suitable algorithm is selected, the model is trained on the training data, and its performance is evaluated using testing data. The model is then tuned to improve its accuracy and make reliable predictions for the given problem.

Activity 2.1: Decision Tree Classifier

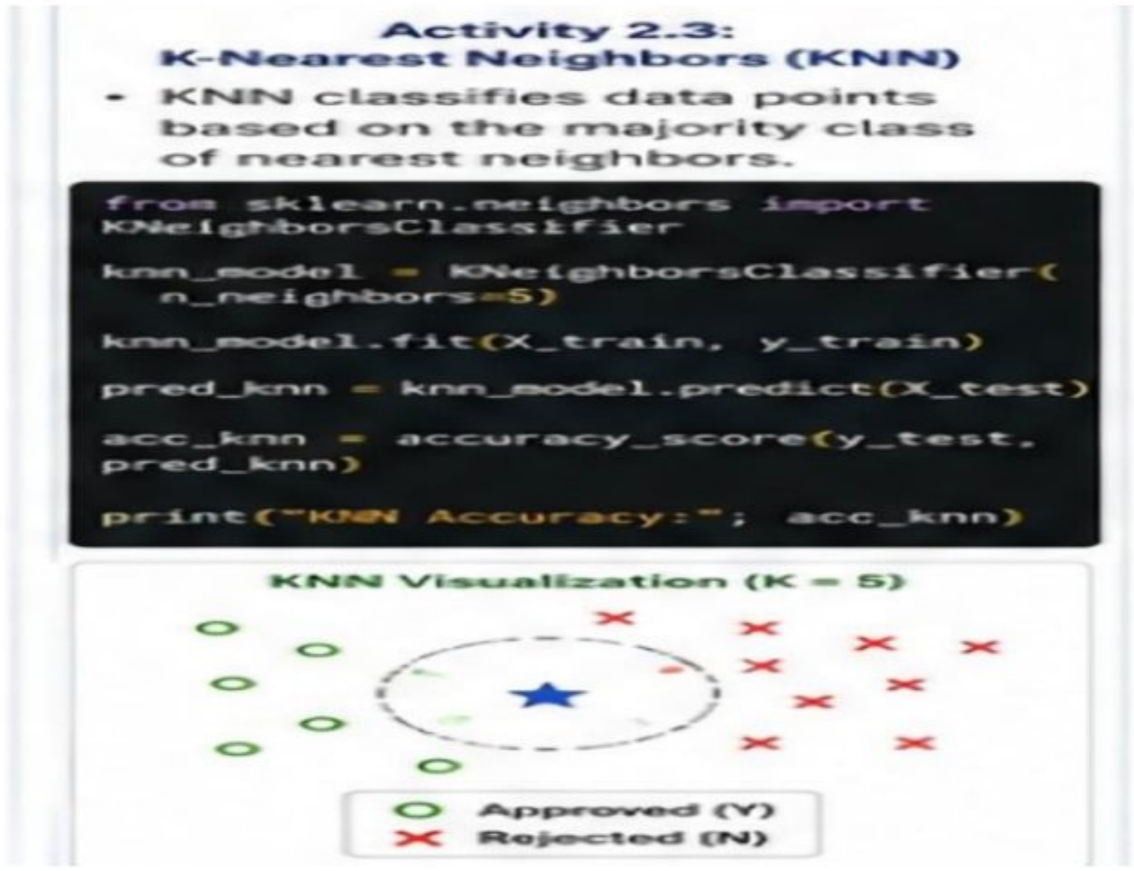
Explanation

The Decision Tree algorithm is implemented as the first machine learning model for predicting loan approval. It learns decision rules from applicant features such as income, credit history, education, and loan amount by creating a tree-like structure. The model is trained using the training dataset and tested using the testing dataset to measure its prediction accuracy.

- **Activity 2.2: Random Forest Classifier**
- Explanation
- The Random Forest model is developed by combining multiple Decision Trees. Each tree makes an independent prediction, and the final result is determined by majority voting. This approach improves prediction accuracy and reduces overfitting compared to a single Decision Tree.



- **Activity 2.3: K-Nearest Neighbors (KNN)**
- Explanation
- The KNN algorithm predicts loan approval by comparing a new applicant with the most similar applicants in the training dataset. The prediction is based on the majority class among the nearest neighbors. This model is evaluated to compare its performance with the other algorithms.



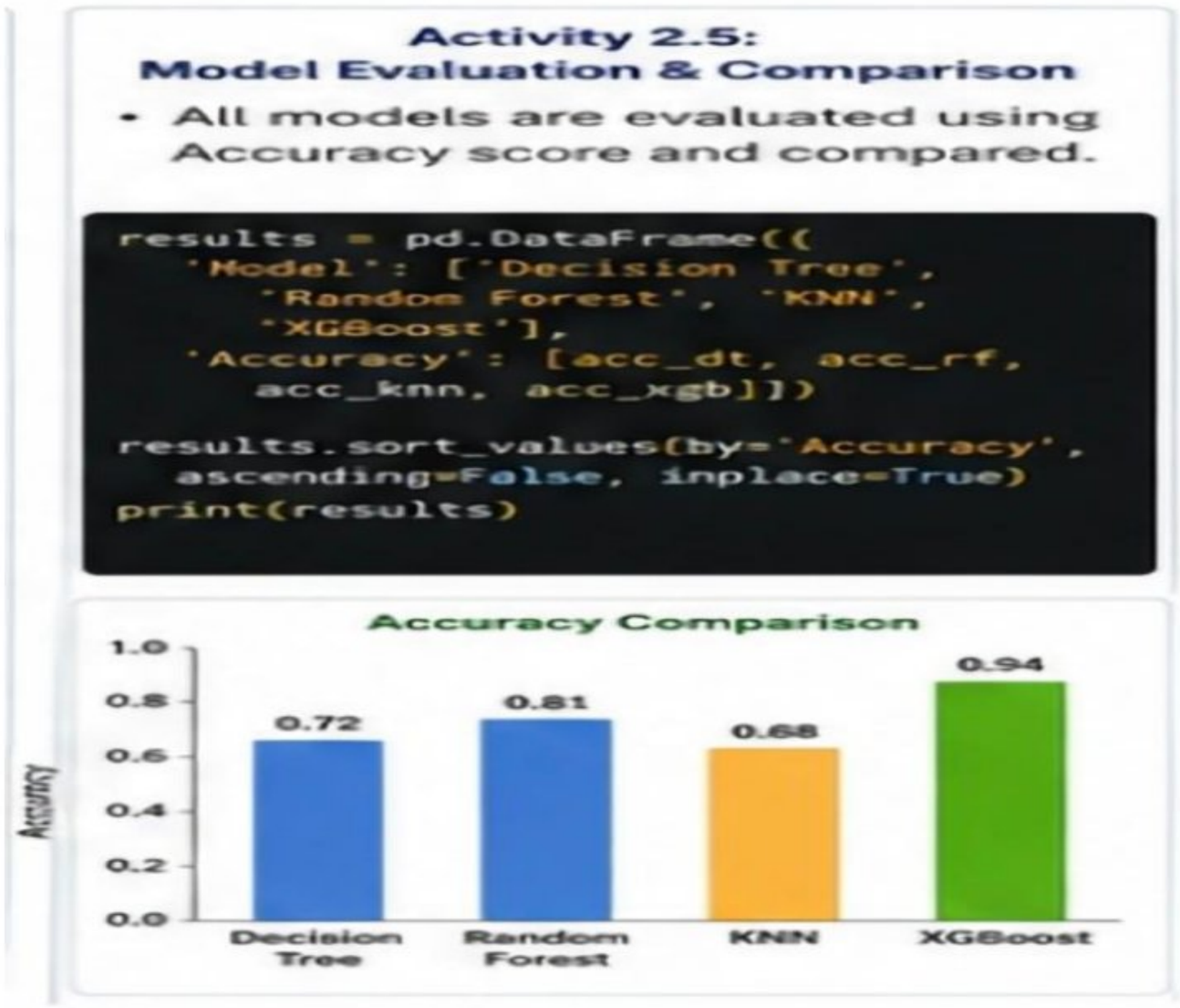
- **Activity 2.4: XGBoost Classifier**
- Explanation
- The XGBoost algorithm is implemented because of its high prediction performance. It combines multiple weak learners into a strong predictive model using gradient boosting. After training, it achieved the highest accuracy among all models and was selected as the final model for the Smart Lender application.

Activity 2.5: Model Evaluation and Comparison

Explanation

All four machine learning models are evaluated using performance metrics such as

Accuracy, Precision, Recall, and F1-Score. Their results are compared, and the best-performing model is selected for deployment. XGBoost achieved the highest performance and became the final prediction model.



Milestone 3:Model Training and Flask Backend Development

Model Training:

Model training is the process of teaching a machine learning model using training data so it can learn patterns and make accurate predictions on new data.

Flask Backend Development:

Flask is a lightweight Python web framework used to build the backend of an application. It connects the trained machine learning model with the user interface, processes user input, and returns prediction results.

Activity 3.1: Save the Trained Model

Explanation

After identifying XGBoost as the best-performing model, it is saved using the Pickle library. Saving the trained model allows the Flask application to load it directly without retraining every time the application starts.

Activity 3.1: Save the Trained Model

- The best model (XGBoost) is saved using Pickle so that it can be loaded in the Flask application for predictions.

```
# Save the trained XGBoost model
import pickle
with open('model.pkl', 'wb') as file:
    pickle.dump(xgb_model, file)
print("Model saved as model.pkl")
```

SMART_LENDER/

- app.py
- model.pkl
- requirements.txt
- templates/
- static/

model.pkl Properties

Type:	Pickle File
Size:	278 KB
Modified:	Today, 10:30 AM

Activity 3.2: Develop the Flask Application (app.py)
Explanation
The Flask framework is used to develop the backend of the Smart Lender application. It receives user inputs from the web interface, loads the trained model, performs prediction, and returns the loan approval result to the frontend.

Activity 3.2: Develop the Flask Application (app.py)

- The Flask application is created. It loads the trained model and defines the necessary routes.

```
from flask import Flask, render_template,
request, jsonify

import pickle
import pandas as pd

app = Flask(__name__)

# Load the trained model
model = pickle.load(open('model.pkl', 'rb'))

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
```

app.py (partial code)

Project Structure

- SMART_LENDER/
  - static/
    - css/
    - js/
    - images/
  - templates/
    - index.html
  - app.py
  - model.pkl
  - requirements.txt

- **Activity 3.3: Create Prediction Route**
- Explanation
- A prediction route is implemented to accept applicant details submitted through the web application. The backend processes the input data, loads the trained model, predicts loan approval status, and returns the prediction result.

Activity 3.3: Create Prediction Route

- A prediction route is created to accept user inputs, process the data, make prediction and return the result.

```python
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()

    # Convert input to DataFrame
    input_df = pd.DataFrame([data])

    # Make prediction
    prediction = model.predict(input_df)[0]

    # Map result
    result = 'Approved' if prediction == 1 else 'Rejected'

    return jsonify({'prediction': result})
```

Prediction Route Code

Activity 3.4: Test Backend Functionality

Explanation

The backend application is tested using sample applicant details to ensure the model loads correctly and returns accurate predictions without errors.

Activity 3.4: Test Backend Functionality

- The backend is tested using sample data. The Flask server runs successfully and returns the prediction.

```
Windows PowerShell
PS D:\SMART_LENDER> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Flask Server Running

```
POST http://127.0.0.1:5000/predict
{
    "prediction": "Approved"
}
```

Sample Prediction Output

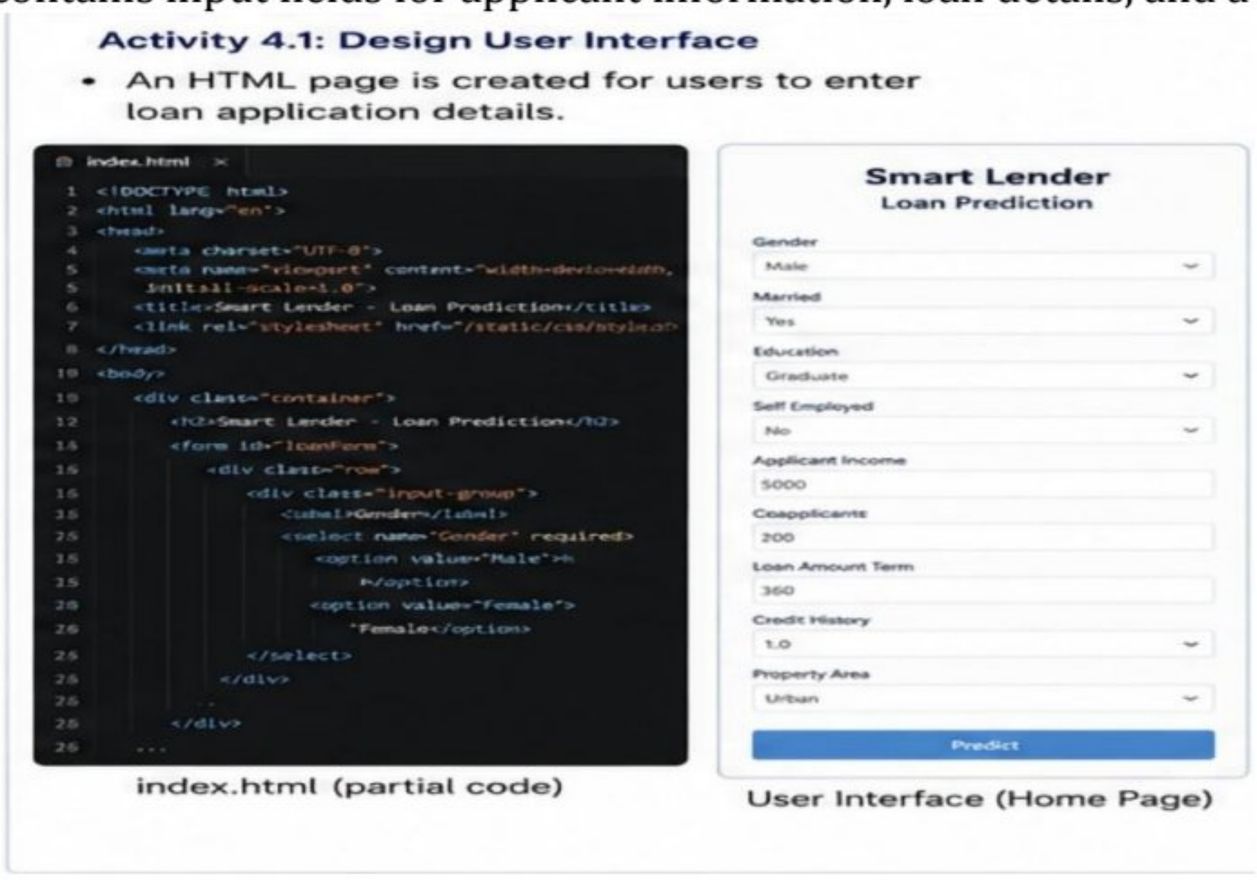
Milestone 4: Frontend Development

Frontend Development is the process of creating the user interface (UI) of the application. It focuses on designing responsive, user-friendly, and interactive web pages that allow users to easily interact with the system. Technologies like HTML, CSS, JavaScript, and frontend frameworks are used to build an attractive and functional interface that connects with the backend and displays data effectively.

Activity 4.1: Design User Interface

Explanation

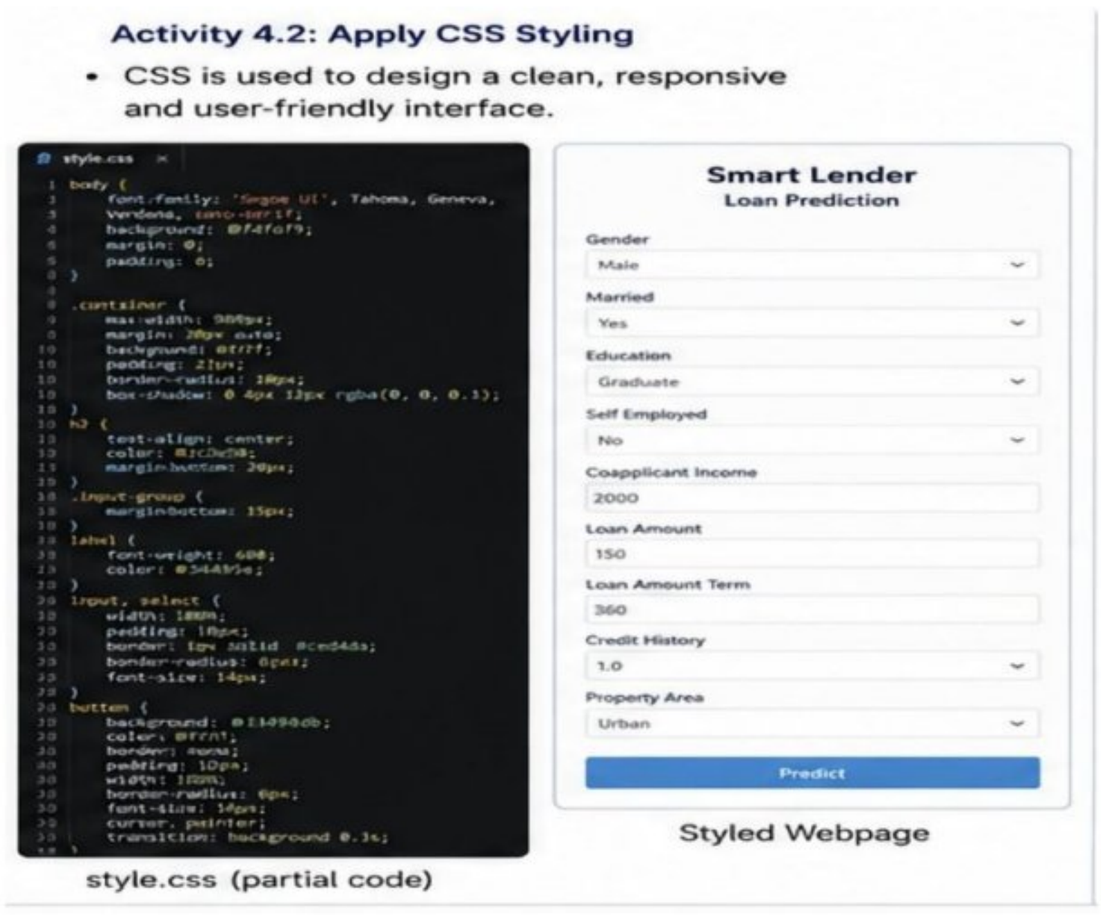
A responsive HTML page is developed to allow users to enter loan application details. The interface contains input fields for applicant information, loan details, and a Predict button.



Activity 4.2: Apply CSS Styling

Explanation

CSS is used to create a clean and attractive interface. Colors, fonts, spacing, and responsive layouts improve the overall user experience.



Activity 4.3: Form Validation

Explanation

JavaScript validates user inputs before sending them to the Flask server. Validation prevents empty fields and incorrect data formats, ensuring reliable predictions.

Activity 4.3: Form Validation

- JavaScript is used to validate inputs before sending data to the server.

```
script.js
document.getElementById('loanForm').addEventListener
  ('submit', function(e) {
      const loanAmount = document.getElementById('loanAmount').value;
      const income = document.getElementById('appIncome').value;

      if(loanAmount === '' || income === '') {
          alert('Please fill all required fields!');
          e.preventDefault();
          return false;
      }
      if(loanAmount <= 0) {
          alert('Loan Amount must be greater than 0');
          e.preventDefault();
          return false;
      }
});
```

script.js (partial code)

Validation Message

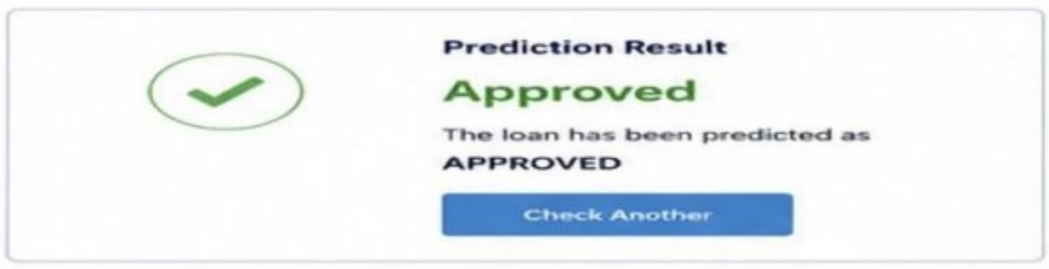
Activity 4.4: Display Prediction Result

Explanation

After processing the application, the predicted loan status is displayed on the webpage. The result indicates whether the loan is approved or rejected based on the applicant's information.

Activity 4.4: Display Prediction Result

- After successful prediction, the result is displayed to the user on the webpage.



Prediction Result - Approved

Prediction Result - Rejected

Milestone 5: Deployment

Deployment is the final stage of a project where the developed application is made available for users. The trained machine learning model, backend, and frontend are hosted on a server or cloud platform. After deployment, the application is tested to ensure it works correctly, is secure, and performs reliably for end users.

Activity 5.1: Configure Project

Environment

Explanation

All required Python libraries are installed using the requirements.txt file, and the project environment is configured for deployment

- Then open “Command Prompt”

- requirements.txt

```
Flask==2.2.5
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.4.2
joblib==1.4.2
python-dotenv===1.0.1
```

- Terminal installation

```
user@laptop:~/smart-lender$ pip install -r requirements.txt
Collecting Flask==2.2.5
Collecting pandas==2.2.2
Collecting numpy==1.26.4
Collecting scikit-learn==1.4.2
Collecting joblib==1.4.2
Collecting python-dotenv==1.0.1
Installing collected packages: ...
Successfully installed Flask-2.2.5 pandas-2.2.2 numpy-1.26.4
scikit-learn-1.4.2 joblib-1.4.2 python-dotenv-1.0.1
user@laptop:~/smart-lender$
```

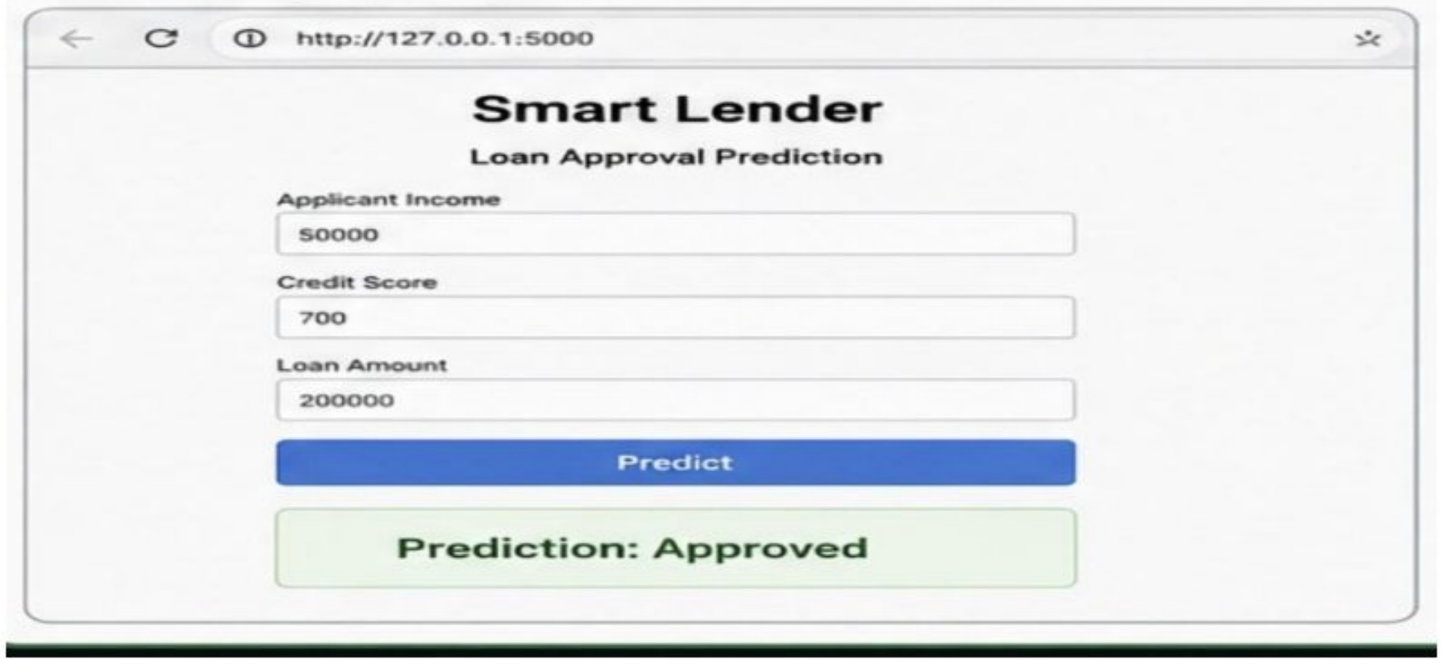
Activity 5.2: Run Flask Application

- Explanation
- The Smart Lender application is executed locally
- using the Flask development server. The application is
- tested by opening the local server URL in a web
- browser.

- Flask terminal

```
user@laptop:~/smart-lender$ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use
  it in a production deployment.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 123-456-789
```

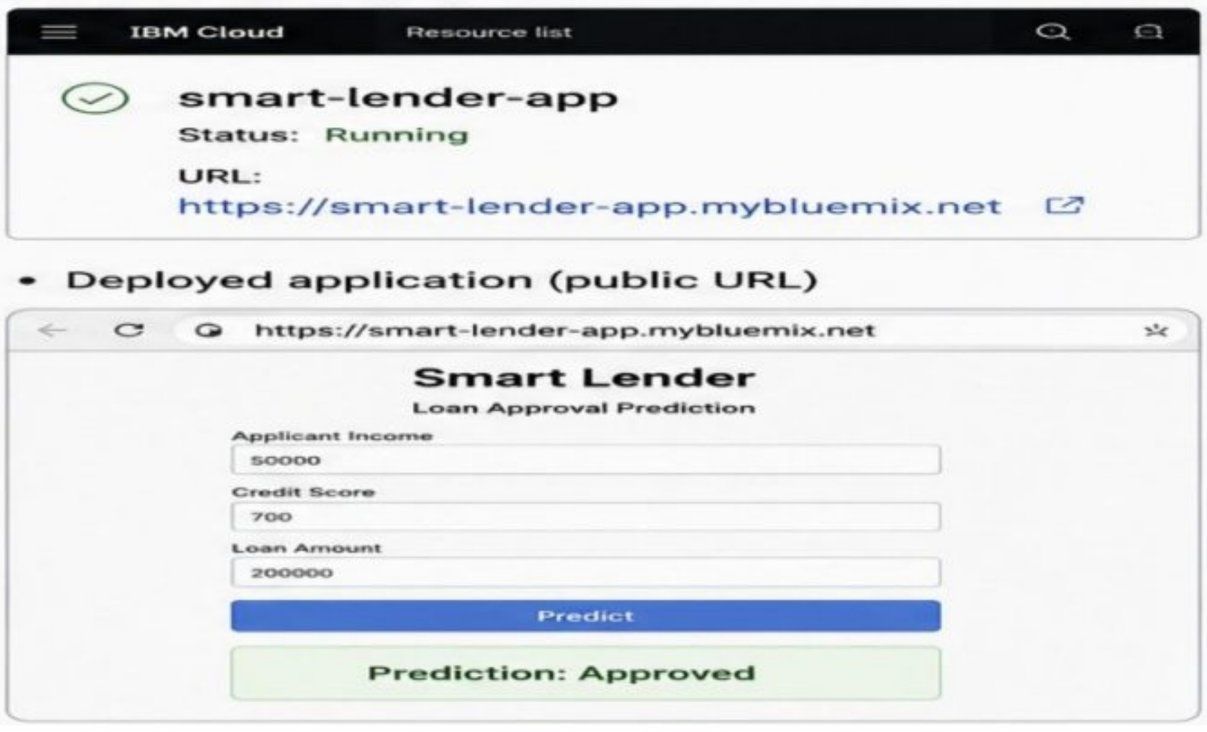
- Running localhost



Activity 5.3: Deploy the Application

The completed Smart Lender application is deployed to IBM Cloud (or your deployment platform). The deployed application can be accessed through a oublic URL for real-time loan prediction

- IBM Cloud deployment

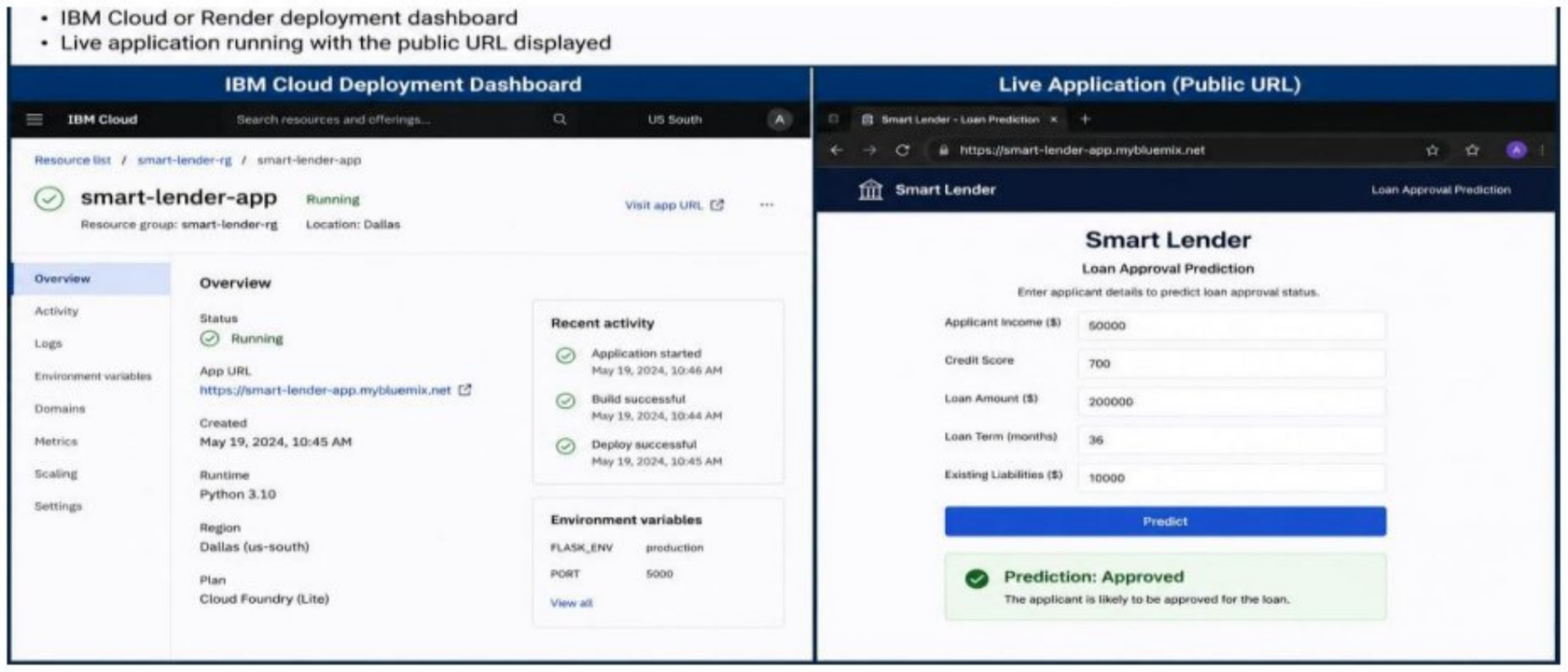


- Deployed application (public URL)

Activity 5.4: Deploy using IBM Cloud (or Render if you used Render)

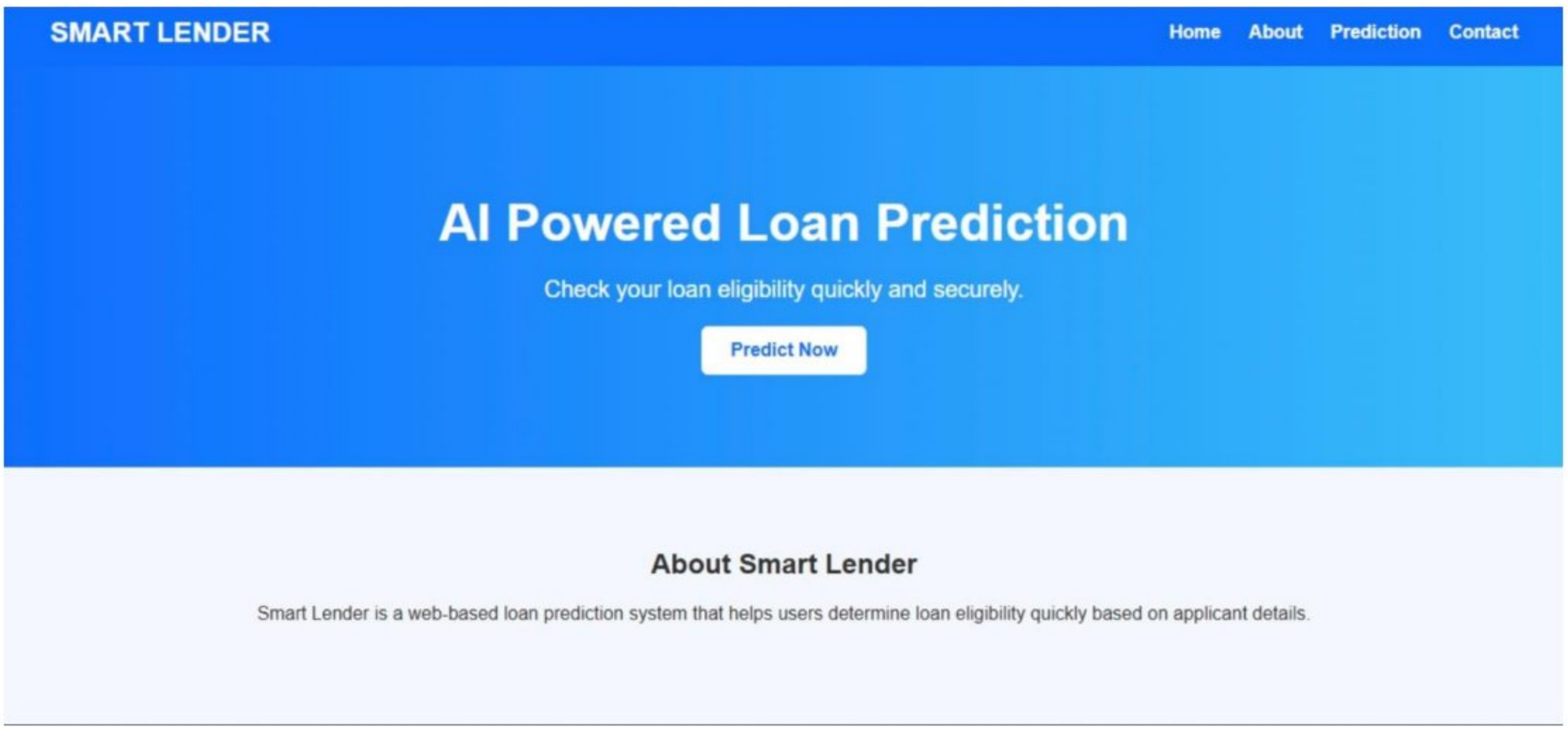
Explanation:

The Smart Lender application is deployed to a cloud platform such as IBM Cloud or Render. After deployment, the application becomes accessible through a public URL, allowing users to access and test the loan prediction system from any device with an internet connection.



Exploring the Website's Web Pages:

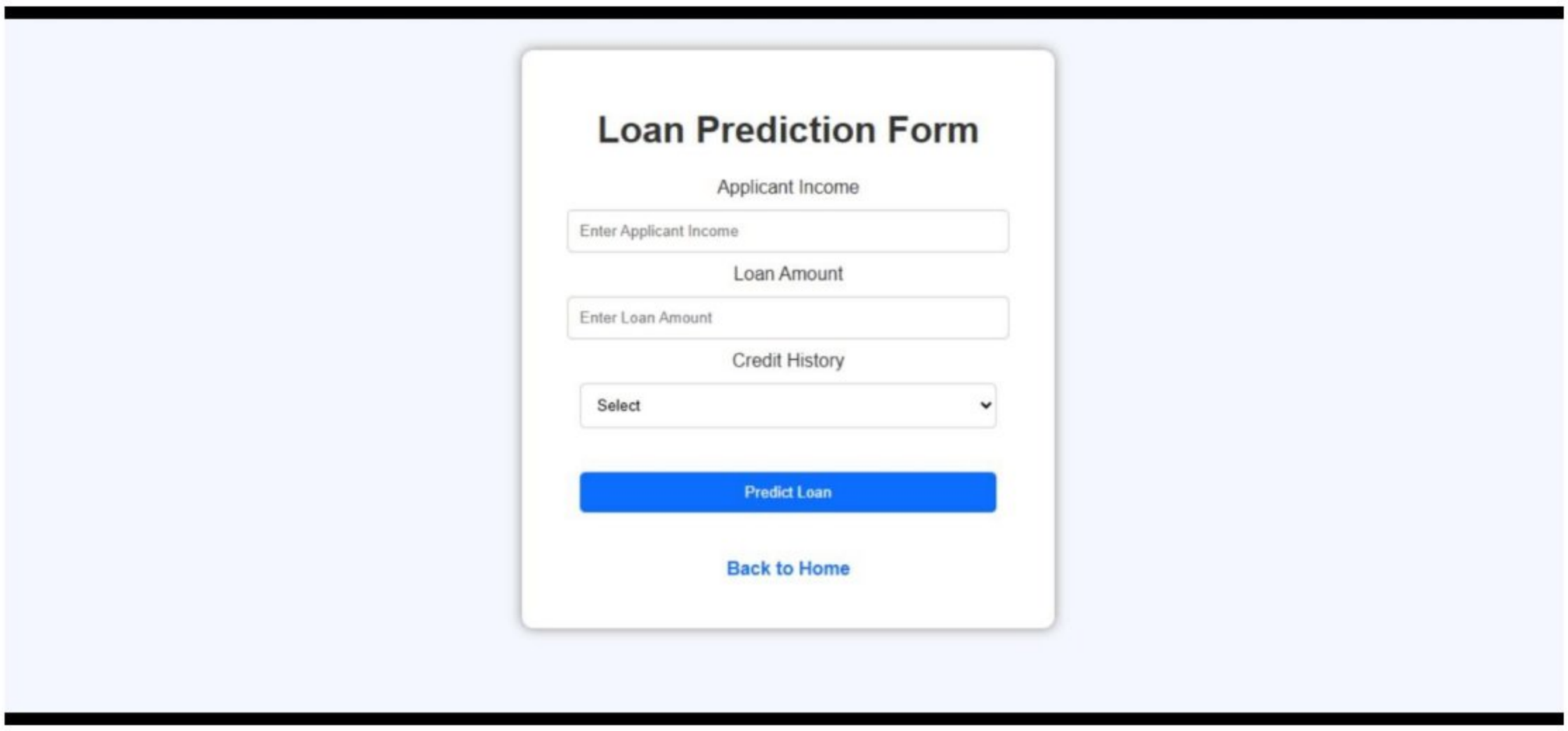
Home / Generator Page:



Description:

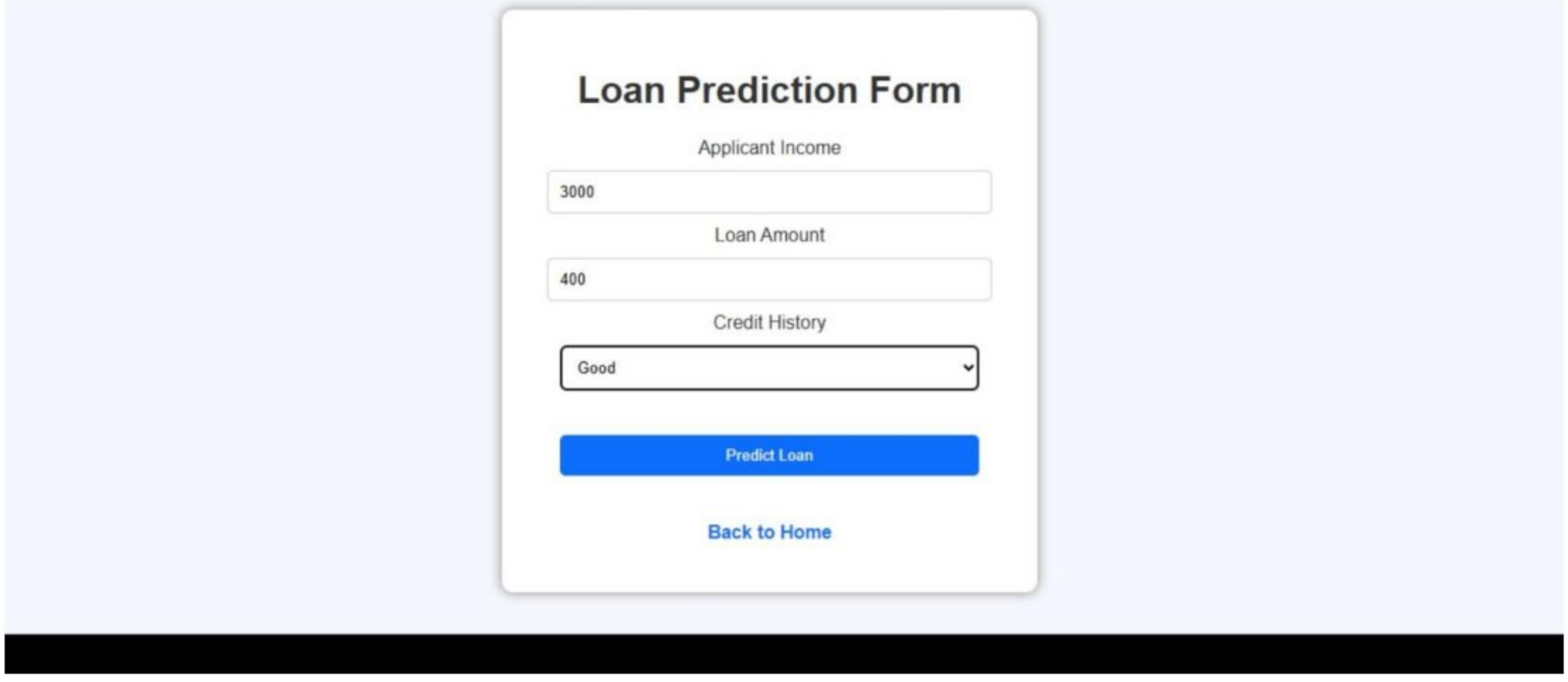
The Smart Lender home page is successfully deployed and accessible through a web browser. It provides a user-friendly interface with an AI-powered loan prediction system, allowing users to check their loan eligibility quickly and securely. The homepage includes a navigation menu, a "Predict Now" button, and an overview of the Smart Lender application, demonstrating that the deployment was completed successfully and the application is ready for public use.

INPUT SECTION :



Description:

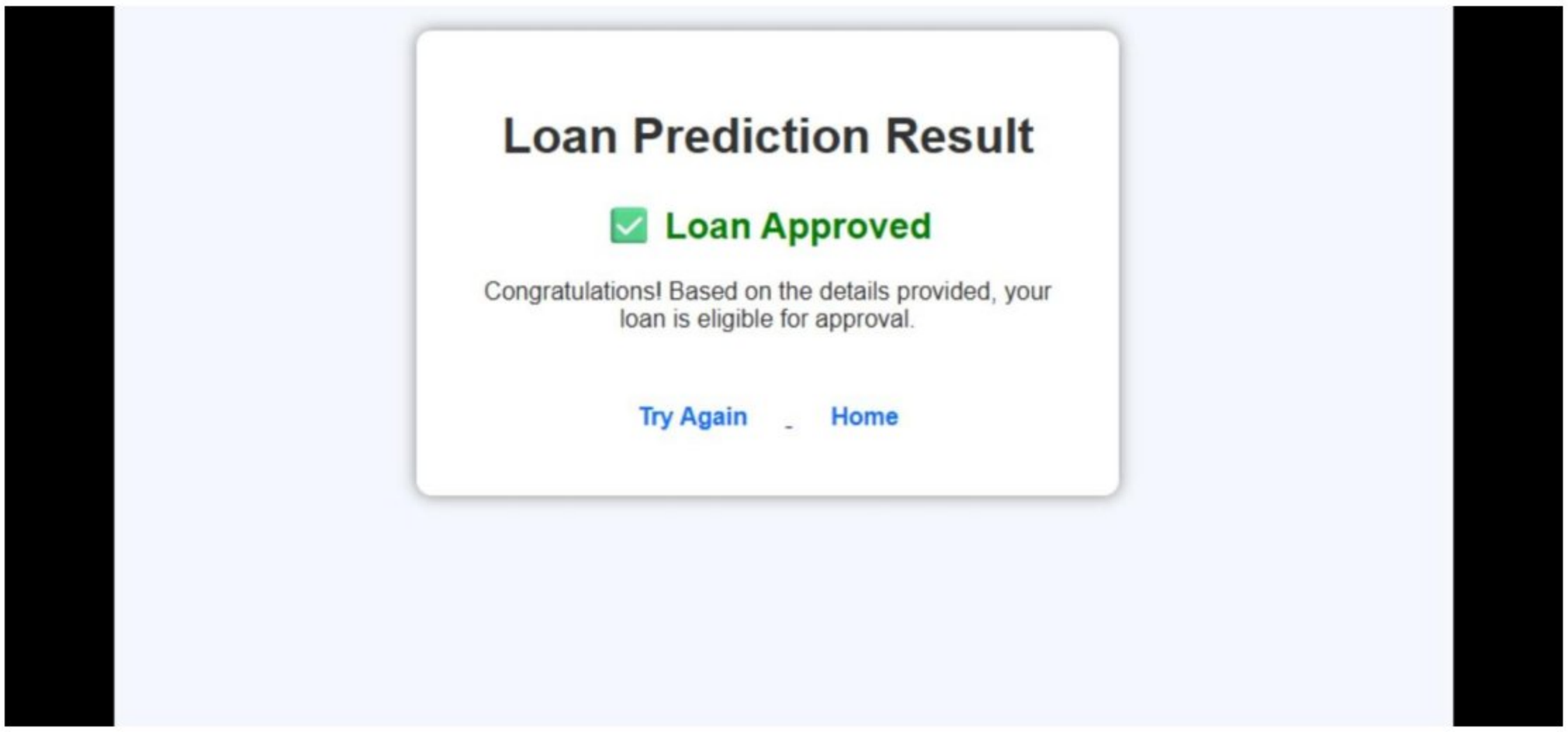
The Loan Prediction Form provides a simple interface for users to enter the required loan application details, such as applicant income, loan amount, and credit history. After filling in the information, users can click the "Predict Loan" button to receive the loan eligibility prediction generated by the trained machine learning model. The form also includes a "Back to Home" option for easy navigation.



Description:

The Loan Prediction Form is filled with the applicant's details, including income, loan amount, and credit history. These inputs are provided to the machine learning model for analysis. When the "Predict Loan" button is clicked, the system processes the entered data and predicts whether the applicant is eligible for a loan. This demonstrates the successful interaction between the user interface and the prediction model.

Results Section:



Description:

The Loan Prediction Result page displays the outcome generated by the machine learning model after processing the applicant's details. In this case, the result indicates "Loan Approved", confirming that the applicant meets the eligibility criteria. The page also provides "Try Again" and "Home" options, allowing users to perform another prediction or return to the application's home page.

Conclusion:

CaptionAI is a generative AI platform built with Flask and styled with a glassmorphism UI that streamlines social media content creation. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate tailored captions, hashtags, and calls-to-action for Instagram and LinkedIn across professional, casual, and promotional tones. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost productivity for marketers and creators, with potential for future scalability. Looking ahead, CaptionAI offers significant opportunities for expansion, such as integrating multi-language support, adding image analysis capabilities for visual-context generation, and implementing user authentication to save generation history. By continuously refining the underlying model prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive social media management assistant for teams of all sizes.